Study Notes: Days 29–35 — ASP.NET Core + Web API

These notes follow Month 2, Week 1 of the roadmap, which is **Week 5 overall**. C# examples are independent unless identified as application fragments. Security examples illustrate configuration and tradeoffs; use your identity provider's documented settings and verify the resulting behavior before production use.

Day 29 — REST principles

Month 2 — Full-Stack Depth, Week 1 — ASP.NET Core + Web API (Week 5 overall).

Goal: Explain REST as an architectural style, design resource-oriented HTTP APIs, and reason about caching, statelessness, and tradeoffs.

Resources, representations, and a uniform interface

A **resource** is a concept identified by a URI, such as a customer or a deposit; a **representation** is data describing its state, such as JSON. A resource can have multiple representations, potentially negotiated using HTTP headers. For example, `/api/deposits/42` identifies a deposit; the JSON returned by `GET` is a representation, not the resource itself.

REST's uniform interface includes identifying resources, manipulating them through representations, self-descriptive messages, and hypermedia as the engine of application state (**HATEOAS**): representations provide links or controls that let clients discover available transitions. Many practical "REST APIs" use resource-oriented HTTP and JSON but do not implement hypermedia fully; call them RESTful in the common industry sense while recognizing the distinction rather than claiming that JSON plus URLs alone satisfies all REST constraints.

Use nouns for stable resources and standard HTTP semantics where they fit: `GET /api/deposits/42` retrieves a representation, while `POST /api/deposits` can create one. Some domain actions do not map naturally to ordinary CRUD. An explicit action endpoint can be clearer than forcing an unnatural resource model, but document its semantics, retry behavior, authorization, and effects.

Statelessness, cacheability, and layers

In REST, each request contains the information needed for the server to understand and process it; the server does not rely on conversational client session state stored between requests. A server may still store durable business data, and an authentication token is not the same thing as server-side session state. Statelessness helps horizontal scaling and independent request handling, but may increase per-request overhead and does not by itself make operations safe or idempotent.

Responses should indicate whether they may be cached. `Cache-Control` controls caching behavior; `ETag` and conditional requests such as `If-None-Match` can avoid transferring an unchanged representation. Personalized or sensitive responses need appropriate restrictions such as `Cache-Control: no-store`; do not assume an authenticated response is safe for a shared cache. Caching improves latency and load only if freshness, invalidation, and authorization boundaries are correct.

A layered system allows intermediaries such as proxies or gateways between client and server. Clients need not know all layers, but proxies introduce concerns such as forwarded headers, cache behavior, timeouts, and

observability. REST also includes client-server separation and optional code-on-demand. These constraints do not mandate microservices, JSON, or a particular database.

API design tradeoffs

Design representations around consumer needs without coupling them directly to internal database entities. Specify pagination, filtering, ordering, error formats, versioning, and authorization. A stable tie-breaker matters for paginated data; offset pagination may be simple but can skip or repeat rows when records change, while cursor pagination has different complexity and consistency tradeoffs.

HTTP semantics and resource design support interoperability, but distributed systems still need explicit handling for concurrency, retries, rate limits, and partial failure. For example, a conditional update using `If-Match` and an ETag can prevent overwriting a newer representation, provided the ETag and update are checked atomically.

Code example

This minimal API is a self-contained resource representation example for a `dotnet new web` project; it does not persist data or implement hypermedia:

```
var builder = WebApplication.CreateBuilder(args);
var app = builder.Build();

app.MapGet("/api/deposits/{id:int}", (int id, HttpContext context) =>
{
    context.Response.Headers.CacheControl = "no-store";
    return id == 42
        ? Results.Ok(new { id = 42, currency = "INR", principal = 10000m })
        : Results.NotFound();
});

app.Run();
```

The response describes a resource in JSON. `no-store` illustrates a conservative caching choice for sensitive data, not a universal rule for all APIs. Authorization and persistence would still be needed for a real deposit.

DSA problem: Majority element

Problem: Given an integer array, find an element appearing **more than half** the time, or return `null` if none exists.

Approach: Boyer–Moore voting pairs off different values. Maintain a candidate and a balance: when the balance is zero, choose the current value; increase it for a matching value and decrease it otherwise. If a majority exists, it survives the cancellations. Since this problem does not guarantee a majority, count the final candidate in a second pass and verify it occurs more than `n / 2` times.

Complexity: O(n) time and O(1) additional space.

```
using System;

Console.WriteLine(FindMajority(new[] { 2, 1, 2, 2 })); // 2
Console.WriteLine(FindMajority(new[] { 1, 2, 3 }) is null); // True

static int? FindMajority(int[] values)
{
    if (values.Length == 0) return null;

    int candidate = 0;
    int balance = 0;
    foreach (int value in values)
    {
        if (balance == 0) candidate = value;
        balance += value == candidate ? 1 : -1;
    }

    int count = 0;
    foreach (int value in values)
        if (value == candidate) count++;
    return count > values.Length / 2 ? candidate : null;
}
```

Edge cases: Empty input has no majority; one element is a majority; exactly half is **not** a majority; negative numbers and zero work. Without a guaranteed majority, omitting the verification pass can return an incorrect answer.

Likely interview questions

1. What is the difference between a resource and its representation?
2. Which REST constraints are often omitted from ordinary JSON APIs?
3. Does statelessness mean the server cannot store business data?
4. How would you cache a sensitive, user-specific response safely?
5. How can `ETag` and `If-Match` help prevent lost updates?
6. When is a domain action endpoint clearer than forcing everything into CRUD?
7. Why can offset pagination be unstable under concurrent changes?
8. Why must Boyer–Moore's candidate be verified when a majority is not guaranteed?

Common pitfalls / misconceptions

- Equating REST with JSON and noun-based URLs while ignoring HTTP semantics and hypermedia.
- Treating statelessness as a ban on persistent data.
- Caching personalized data without considering shared caches and authorization.
- Returning a Boyer–Moore candidate without confirming the majority condition.

Cheat-sheet recap

- Resource = identified concept; representation = transferred description.
- Stateless requests, cacheability, uniform interface, and layered system are key REST constraints.

- Use conditional requests and explicit cache policy when appropriate.
- Majority means **more than** `n / 2`; verify the voting candidate if existence is not guaranteed.

Day 30 — HTTP methods / status codes

Month 2 — Full-Stack Depth, Week 1 — ASP.NET Core + Web API (Week 5 overall).

Goal: Choose methods and status codes according to their defined semantics, including safe retries, preconditions, and failure handling.

HTTP methods

`GET` retrieves a representation and is **safe**: the client does not request a state-changing operation. `HEAD` has the semantics of `GET` but no response body. `OPTIONS` describes communication options and is also used in CORS preflight. Safe methods can still have incidental effects such as access logs.

`PUT` creates or replaces the target resource according to the API's contract and is **idempotent**: repeating the same request has the same intended effect as performing it once. `DELETE` is also idempotent in intended effect, even if the first response is 204 and a later response is 404. `POST` asks the target to process a request, often to create a subordinate resource; it is not inherently idempotent. `PATCH` applies partial modification and is not guaranteed to be idempotent—the patch format and operation determine that behavior.

Idempotency concerns intended server effect, not identical responses, absence of logs, or automatic safety of every implementation. A retry of a non-idempotent operation after a timeout may duplicate work because the original may already have succeeded. An idempotency key with durable, correctly scoped deduplication can make a particular operation retryable; define its lifetime, request matching, and behavior under concurrent duplicates.

Status code families and common choices

- **2xx:** `200 OK` returns a successful response; `201 Created` commonly includes `Location` for a new resource; `202 Accepted` means processing is accepted but not necessarily completed; `204 No Content` has no response body.
- **3xx:** `304 Not Modified` can answer a conditional `GET` or `HEAD` when the cached representation remains valid; redirection codes have distinct method-preservation behavior.
- **4xx:** `400 Bad Request` commonly covers malformed or invalid request input; `401 Unauthorized` means authentication is absent or invalid and normally includes a `WWW-Authenticate` challenge; `403 Forbidden` means the request is understood but access is refused; `404 Not Found` means the resource is absent or the server elects not to disclose it.
- **Conflicts and preconditions:** `409 Conflict` can describe a conflict with current resource state; `412 Precondition Failed` is appropriate when a supplied condition such as `If-Match` fails. `422 Unprocessable Content` may be used for semantically invalid content where an API chooses that convention. Be consistent and document the contract.
- **5xx:** `500 Internal Server Error` represents an unexpected server failure; `503 Service Unavailable` indicates temporary unavailability and may include `Retry-After`. Avoid exposing internal details in error bodies.

`429 Too Many Requests` indicates rate limiting and may also provide `Retry-After`. Do not automatically retry every 4xx or 5xx: consider the operation's idempotency, whether the failure is transient, backoff, jitter,

cancellation, and downstream capacity.

Conditional requests and concurrency

An ETag represents a version of a representation. A client can send `If-Match` on an update to require a matching current version; the server must check the condition atomically with the update. A failed supplied precondition normally produces 412. A service may require a precondition and use `428 Precondition Required` if it is missing. Strong versus weak ETag semantics matter: `If-Match` uses strong comparison.

Code example

This minimal API demonstrates response shapes without implementing storage:

```
var builder = WebApplication.CreateBuilder(args);
var app = builder.Build();

app.MapGet("/api/items/{id:int}", (int id) =>
    id == 1 ? Results.Ok(new { id = 1, name = "Example" }) : Results.NotFound());

app.MapPost("/api/items", (CreateItem request) =>
{
    if (string.IsNullOrWhiteSpace(request.Name))
        return Results.BadRequest(new { error = "Name is required." });
    const int id = 2;
    return Results.Created($"/api/items/{id}", new { id, request.Name });
});

app.MapDelete("/api/items/{id:int}", (int id) =>
    id == 1 ? Results.NoContent() : Results.NotFound());

app.Run();

record CreateItem(string? Name);
```

The sample uses fixed values and does not persist changes. A real creation response must refer to a retrievable resource, and a real delete must actually change persistent state.

Likely interview questions

1. What is the difference between a safe method and an idempotent method?
2. Why can a repeated DELETE return a different status while remaining idempotent?
3. Why is POST not inherently safe to retry after a timeout?
4. How do PUT and PATCH differ, and is PATCH always idempotent?
5. When would you use 201, 202, or 204?
6. What is the difference between 401 and 403?
7. When would a failed update return 409 versus 412?
8. What does an idempotency key need to guarantee under concurrent retries?

Common pitfalls / misconceptions

- Assuming idempotent means every response must be identical.
- Putting a state-changing operation behind GET and relying on clients not to retry or prefetch it.
- Returning 200 for every failure or including a body with 204.
- Retrying non-idempotent requests without a deduplication design.

Cheat-sheet recap

- GET/HEAD are safe; PUT and DELETE are idempotent in intended effect; POST and PATCH are not inherently idempotent.
- 201 = created; 202 = accepted, not finished; 204 = success without a body.
- 401 = authentication challenge; 403 = refused; 409 = conflict; 412 = failed precondition.
- Retry only with known semantics, bounded backoff, and appropriate deduplication.

Day 31 — Authentication: JWT, bearer tokens, claims

Month 2 — Full-Stack Depth, Week 1 — ASP.NET Core + Web API (Week 5 overall).

Goal: Explain how an API authenticates a bearer access token, validates JWTs, and turns trusted claims into an identity.

Authentication and bearer tokens

Authentication establishes who or what is making a request; authorization decides what that identity may do. A bearer token is a credential: anyone who possesses it can generally present it, typically using `Authorization: Bearer <token>`. Protect it in transit with HTTPS, avoid putting it in URLs or logs, and choose storage appropriate to the client and threat model.

OAuth access tokens can be opaque or JWTs; a JWT is a token format, not a synonym for OAuth, authentication, or encryption. OpenID Connect adds identity-related semantics on top of OAuth; an ID token is intended for the client and should not be accepted by an API as an access token just because it is a JWT. The API should validate that the access token was issued for that API and is of the expected kind for the issuer and deployment.

JWT structure and validation

A signed JWT commonly has base64url-encoded header, payload, and signature segments. Encoding is not encryption: claims in an ordinary signed JWT can be read by anyone holding it. The signature protects integrity and proves possession of the signing key, but trusting a token requires more than decoding it.

An API must validate the signature against keys it trusts for the expected issuer, restrict acceptable algorithms, verify issuer and audience, enforce expiry and other relevant lifetime rules, and apply an appropriate clock skew. Use trusted discovery metadata and signing-key rotation mechanisms where supported; never trust an arbitrary key location merely because a token header names it. A token signed by a trusted key can still be inappropriate if its issuer, audience, type, or intended purpose is wrong.

Short-lived access tokens limit exposure but do not automatically provide immediate revocation. A deployment may use introspection for opaque tokens, revocation strategies, or other controls depending on the issuer and risk. Refresh tokens require especially careful storage and rotation; APIs should not accept a refresh token as an access token.

Claims and ASP.NET Core

Claims are statements about a subject or token, such as `sub`, issuer, audience, roles, or permissions. Their names and formats are issuer-specific; a scope may be a space-separated claim rather than one claim per scope. Validate the token before relying on claims, distinguish identity from permissions, and avoid storing sensitive information unnecessarily in token payloads.

ASP.NET Core authentication middleware validates credentials using a configured scheme and produces a `ClaimsPrincipal`. Configure authorization separately and run authentication before authorization. Claim mapping can change the claim names exposed to application code; configure and test it intentionally. Claims are a point-in-time assertion, so membership changes, token lifetime, and permission revocation are architectural concerns.

Code example

This `Program.cs` requires a real HTTPS OpenID Connect/OAuth issuer, an access-token audience for this API, and the `Microsoft.AspNetCore.Authentication.JwtBearer` package compatible with the project's .NET version. The identity provider must issue tokens using the configured issuer and RS256; change the explicit algorithm allowlist only to match your trusted provider.

```
using System.Security.Claims;
using Microsoft.AspNetCore.Authentication.JwtBearer;
using Microsoft.IdentityModel.Tokens;

var builder = WebApplication.CreateBuilder(args);
string authority = builder.Configuration["Jwt:Authority"]
    ?? throw new InvalidOperationException("Jwt:Authority is required.");
string issuer = builder.Configuration["Jwt:Issuer"]
    ?? throw new InvalidOperationException("Jwt:Issuer is required.");
string audience = builder.Configuration["Jwt:Audience"]
    ?? throw new InvalidOperationException("Jwt:Audience is required.");

builder.Services
    .AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
    .AddJwtBearer(options =>
    {
        options.Authority = authority;
        options.RequireHttpsMetadata = true;
        options.MapInboundClaims = false;
        options.TokenValidationParameters = new TokenValidationParameters
        {
            ValidateIssuer = true,
            ValidIssuer = issuer,
            ValidateAudience = true,
            ValidAudience = audience,
            ValidateIssuerSigningKey = true,
            RequireSignedTokens = true,
            ValidateLifetime = true,
            RequireExpirationTime = true,
            ValidAlgorithms = new[] { SecurityAlgorithms.RsaSha256 },
            ClockSkew = TimeSpan.FromMinutes(1)
```

```
        };
    });
builder.Services.AddAuthorization();

var app = builder.Build();
app.UseAuthentication();
app.UseAuthorization();
app.MapGet("/me", (ClaimsPrincipal user) =>
    Results.Ok(new { subject = user.FindFirst("sub")?.Value }))
    .RequireAuthorization();
app.Run();
```

Supply `Jwt__Authority`, `Jwt__Issuer`, and `Jwt__Audience` using trusted environment configuration. Discovery and signing-key availability depend on the provider. This snippet is not a token issuer, does not configure client login, and does not itself enforce resource-level permissions.

DSA problem: Subarray Sum = K

Problem: Given an integer array and integer `k`, return the number of contiguous, nonempty subarrays whose sum equals `k`. Values may be positive, negative, or zero.

Approach: Let `prefix` be the sum through the current index. Any earlier prefix equal to `prefix - k` starts a qualifying subarray. Store the frequency of each prefix in a dictionary, initially `{ 0: 1 }` so subarrays starting at index zero are counted. Add the previous frequency before recording the current prefix to avoid counting an empty subarray.

Complexity: Expected O(n) time and O(n) extra space. Use `long` for prefix sums and the answer; the number of qualifying subarrays can exceed `int.MaxValue`.

```
using System;
using System.Collections.Generic;

Console.WriteLine(CountSubarrays(new[] { 1, -1, 1 }, 1)); // 3
Console.WriteLine(CountSubarrays(new[] { 0, 0 }, 0));     // 3

static long CountSubarrays(int[] values, int k)
{
    var counts = new Dictionary<long, long> { [0] = 1 };
    long prefix = 0;
    long result = 0;

    foreach (int value in values)
    {
        prefix += value;
        if (counts.TryGetValue(prefix - k, out long matches))
            result += matches;
        counts.TryGetValue(prefix, out long count);
        counts[prefix] = count + 1;
    }
```

```
        return result;
    }
```

Edge cases: Empty input returns zero; repeated prefix sums and `k = 0` are handled; negative numbers mean a simple expanding/shrinking positive-only sliding window is not generally valid. This returns a count, not the subarray indices.

Likely interview questions

1. How do authentication and authorization differ?
2. What is the difference between a bearer token, a JWT, and an OAuth access token?
3. Why is decoding a JWT not enough to trust it?
4. Which issuer, audience, lifetime, algorithm, and signing-key checks should an API perform?
5. Why should an API not accept an ID token merely because it is a valid JWT?
6. What are the consequences of bearer-token theft and how can exposure be reduced?
7. Why can token claims become stale, and how does key rotation affect validation?
8. Why does the prefix-sum algorithm initialize the zero prefix with frequency one?

Common pitfalls / misconceptions

- Treating base64url encoding as encryption or accepting an unsigned/unvalidated token.
- Validating signature while ignoring issuer, audience, lifetime, and token purpose.
- Logging tokens or putting them in URLs.
- Using a positive-only sliding window for subarray sums when negatives are allowed.

Cheat-sheet recap

- A bearer token grants access to whoever possesses it; TLS and careful handling matter.
- JWT = format; validate trusted keys, algorithm, issuer, audience, lifetime, and intended token use.
- Claims inform identity and policy but can become stale.
- Subarray sum: count earlier `prefix - k` values, including the initial zero prefix.

Day 32 — Authorization: roles, claims, policies

Month 2 — Full-Stack Depth, Week 1 — ASP.NET Core + Web API (Week 5 overall).

Goal: Enforce permissions at the endpoint and resource levels without confusing an authenticated identity with an authorized operation.

Roles

A role groups permissions or organizational responsibilities, such as `FinanceManager`. Role checks are simple when the role vocabulary is stable and maps well to the operation. In ASP.NET Core, the role claim type must match the identity provider's claim mapping; a role check against the wrong claim type may fail even when the token contains a similarly named field.

Roles can become coarse or proliferate when every combination of tenant, resource, and action becomes a separate role. They do not automatically establish ownership of a specific record. A user with a broad role may still need tenant and resource checks.

Claims

A claim is a statement associated with an identity, such as a tenant identifier, permission, or department. Claims can support more granular decisions than roles, but their meaning, issuer, format, and lifetime must be understood. Only trust claims from a validated identity and ensure the right issuer is authoritative for the claim. Multiple identities and claims can exist in one principal.

A raw claim is not a complete authorization policy. For example, a tenant claim must be compared with the resource's tenant; accepting an owner or tenant ID supplied by the request without checking the authenticated principal risks cross-tenant access. Scope claims are frequently space-separated, so a direct exact-value claim check may not represent the desired scope membership; parse according to the issuer's contract.

Policies and resource-based authorization

A policy groups requirements, such as an authenticated user plus a role or permission claim. ASP.NET Core evaluates requirements through handlers; policies centralize rules and are reusable across endpoints. Multiple requirements in one policy must succeed. When access depends on a specific loaded resource, use resource-based authorization or an equivalent explicit application check after obtaining the resource and before returning or mutating it.

Use least privilege and deny by default for sensitive endpoints. Differentiate 401 for a missing/invalid identity from 403 for an authenticated identity that lacks permission; some APIs intentionally return 404 instead of 403 to avoid revealing a resource's existence. Ensure the choice is consistent and does not create other information leaks. Tokens and cached permissions may be stale; high-risk operations may need fresher checks.

Code example

This is an application fragment: it assumes a real authentication scheme has already been registered and configured as on Day 31. The issuer must provide a separate `permission` claim with the exact value shown; do not assume that format for every provider.

```
using System.Security.Claims;

// Before builder.Build():
builder.Services.AddAuthorization(options =>
{
    options.AddPolicy("FinanceRead", policy =>
        policy.RequireAuthenticatedUser()
              .RequireClaim("permission", "finance.read"));
});

// After builder.Build(), with authentication configured:
app.UseAuthentication();
app.UseAuthorization();
app.MapGet("/api/finance", (ClaimsPrincipal user) =>
    Results.Ok(new { subject = user.FindFirst("sub")?.Value }))
    .RequireAuthorization("FinanceRead");
```

This checks an endpoint-level permission only. For a particular deposit, also verify that the authenticated user may access that resource and tenant. A test principal or a custom development scheme must not be used as production authentication.

Likely interview questions

1. How do roles, claims, and policies relate to one another?
2. Why does authentication not imply permission to read every resource?
3. How can claim mapping cause a role check to fail unexpectedly?
4. How would you enforce tenant ownership for a deposit?
5. When should resource-based authorization be evaluated?
6. How do 401, 403, and intentionally concealed 404 responses differ?
7. Why can a direct `RequireClaim` check be incorrect for a space-separated scope claim?
8. How do you handle revoked permissions while previously issued tokens remain valid?

Common pitfalls / misconceptions

- Treating client-supplied tenant or owner IDs as authoritative.
- Assuming a role grants access to every resource or every tenant.
- Testing only successful authorization and omitting denial cases.
- Assuming all identity providers format role and scope claims identically.

Cheat-sheet recap

- Roles group responsibilities; claims provide assertions; policies combine requirements.
- Validate the identity, then enforce endpoint **and** resource-level authorization.
- Design for least privilege, tenant isolation, claim freshness, and clear denial behavior.

Day 33 — Global exception handling

Month 2 — Full-Stack Depth, Week 1 — ASP.NET Core + Web API (Week 5 overall).

Goal: Convert expected and unexpected failures into consistent, safe HTTP responses while retaining useful diagnostics.

Centralized handling and problem details

Global exception handling sits early in the middleware pipeline so it can observe exceptions from downstream components. It centralizes safe response formatting and logging policy instead of duplicating broad catches in every controller. ASP.NET Core can produce `ProblemDetails` responses with fields such as type, title, status, and a correlation identifier; clients should not depend on internal exception messages or stack traces.

Expected outcomes such as validation failures, missing resources, or an optimistic concurrency conflict can often be returned explicitly by application or endpoint code rather than thrown as exceptional control flow. A known domain exception may be mapped to a documented response when that is the application's chosen contract. Unexpected exceptions should remain visible to operators and normally produce a generic 500 for clients.

Failure boundaries and operational nuance

Do not catch every exception and return a success-shaped result. Log unexpected failures with structured context and the exception at an appropriate boundary, avoiding secrets, raw tokens, and duplicate logs at every layer. Use a trace identifier to connect a safe response to server-side diagnostics. Status codes should reflect the actual class of failure rather than mapping everything to 400 or 404.

An exception after the response has started may not be convertible into a new problem response; the connection may have to be aborted. Streaming responses therefore need particular care. Request cancellation and client disconnects are not necessarily internal server errors; distinguish them from faults according to the host and application behavior. An exception handler cannot roll back a database transaction or undo an external side effect unless the application explicitly manages that boundary.

Code example

This example uses `IExceptionHandler` and `AddProblemDetails` in a modern ASP.NET Core application. It maps one known conflict; unhandled exceptions fall through to the framework's exception handler and problem-details support.

```
using Microsoft.AspNetCore.Diagnostics;

var builder = WebApplication.CreateBuilder(args);
builder.Services.AddProblemDetails();
builder.Services.AddExceptionHandler<ConflictHandler>();

var app = builder.Build();
app.UseExceptionHandler();
app.MapGet("/conflict", () => TriggerConflict());
app.Run();

static string TriggerConflict() => throw new DepositConflictException();

sealed class DepositConflictException : Exception { }

sealed class ConflictHandler : IExceptionHandler
{
    private readonly ILogger<ConflictHandler> logger;
    public ConflictHandler(ILogger<ConflictHandler> logger) => this.logger =
logger;

    public async ValueTask<bool> TryHandleAsync(
        HttpContext context, Exception exception, CancellationToken
cancellationToken)
    {
        if (exception is not DepositConflictException) return false;

        logger.LogWarning("Deposit conflict. TraceId: {TraceId}",
context.TraceIdentifier);
        await Results.Problem(
            statusCode: StatusCodes.Status409Conflict,
            title: "The deposit conflicts with its current state.",
            extensions: new Dictionary<string, object?> { ["traceId"] =
context.TraceIdentifier })
```

```
            .ExecuteAsync(context);
        return true;
    }
}
```

The example deliberately does not expose the exception's details to the client. Built-in diagnostic behavior for handled exceptions can vary by framework version and configuration, so the handler explicitly records the expected conflict. Production code should also test fallback responses, content negotiation, and failures after a response starts.

DSA problem: Valid anagram

Problem: Determine whether two strings contain the same UTF-16 code units with the same frequencies, using case-sensitive ordinal comparison. For example, `listen` and `silent` match.

Approach: Reject differing lengths. Count code units from the first string; subtract counts for the second and fail if a code unit is missing or exhausted. Equal lengths plus successful subtraction establish that all counts match.

Complexity: Expected O(n) time and O(k) additional space for `k` distinct code units.

```
using System;
using System.Collections.Generic;

Console.WriteLine(IsAnagram("listen", "silent")); // True
Console.WriteLine(IsAnagram("rat", "car"));       // False

static bool IsAnagram(string first, string second)
{
    if (first.Length != second.Length) return false;

    var counts = new Dictionary<char, int>();
    foreach (char character in first)
    {
        counts.TryGetValue(character, out int count);
        counts[character] = count + 1;
    }
    foreach (char character in second)
    {
        if (!counts.TryGetValue(character, out int count) || count == 0)
            return false;
        counts[character] = count - 1;
    }
    return true;
}
```

Edge cases: Two empty strings are anagrams; differing lengths are not; repeated characters matter; case and whitespace are significant under this contract. A C# `char` is a UTF-16 code unit, not necessarily a Unicode

scalar or user-perceived character. If the requirement is linguistic or Unicode-aware, specify normalization, case folding, and whether to compare `Rune` values or grapheme clusters.

Likely interview questions

1. Why should exception handling middleware be registered early in the pipeline?
2. What is `ProblemDetails`, and what must not be exposed in a client response?
3. Which failures are better modeled as explicit results rather than exceptions?
4. How would you map a known conflict while preserving unexpected errors as 500s?
5. What happens when an exception occurs after a response has started?
6. How should cancellation or client disconnects differ from application faults?
7. How do transactions and external side effects relate to exception handling?
8. How do you test an anagram solution for repeated characters and Unicode assumptions?

Common pitfalls / misconceptions

- Returning a generic success response after catching an exception.
- Exposing stack traces or tokens in error bodies and logs.
- Assuming middleware can change headers after the response has started.
- Treating UTF-16 code units as identical to all user-perceived characters.

Cheat-sheet recap

- Handle known failures consistently; log unexpected failures and return safe 500 responses.
- Use problem details and trace correlation without exposing internal information.
- Response-started, cancellation, transaction, and streaming boundaries matter.
- Anagram: compare lengths and character frequencies under explicitly defined text semantics.

Day 34 — CORS + HTTPS + security

Month 2 — Full-Stack Depth, Week 1 — ASP.NET Core + Web API (Week 5 overall).

Goal: Explain browser cross-origin controls, transport security, and practical API security without treating any single feature as a complete defense.

CORS

An origin consists of scheme, host, and port. Cross-Origin Resource Sharing lets a server tell a **browser** which origins may read certain cross-origin responses. It is not authentication or authorization, and non-browser clients can ignore CORS entirely. The same-origin policy and CORS primarily govern browser access to responses; some cross-origin requests can still be sent even when the browser will not expose the response.

For certain methods or headers, the browser sends an `OPTIONS` preflight with the intended method and headers. The server must allow the origin, method, and requested headers for the browser to proceed. Configure only known origins and necessary methods/headers; distinguish a failed preflight from an application authorization failure.

Credentialed cross-origin requests require explicit compatible client and server configuration. Do not combine wildcard origin access with credentials; ASP.NET Core rejects the unsafe `AllowAnyOrigin` plus

`AllowCredentials` combination. Cookie-based cross-origin requests also require appropriate cookie attributes and CSRF defenses. CORS is not a substitute for CSRF protection.

HTTPS and proxies

HTTPS uses TLS to protect data in transit and authenticate the server when certificates are validated. It does not prevent an authorized but malicious user from abusing an endpoint, nor does it protect secrets written to logs. Use valid certificates, redirect or reject insecure requests according to deployment design, and apply HSTS appropriately for browser-facing production sites.

When TLS terminates at a trusted reverse proxy, the application may see an internal HTTP connection. Configure forwarded headers only from known proxies or networks, and apply them before middleware that relies on the original scheme. Misconfiguration can cause redirect loops, insecure generated URLs, or trust in spoofed client headers. HTTPS redirection and forwarded-header behavior should be tested in the actual deployment topology.

Defense in depth

Validate input and use parameterized database queries; authenticate callers and enforce resource-level authorization; protect secrets; apply rate limits and request-size limits appropriate to the service; and log security-relevant events without sensitive data. Use least privilege for infrastructure and dependencies. Set appropriate cache policies for sensitive responses, handle dependency vulnerabilities, and test common attack paths.

Cookie authentication requires attention to CSRF because browsers attach cookies automatically. Bearer tokens in an `Authorization` header have different CSRF and XSS tradeoffs; storing credentials in browser-accessible storage increases the consequences of XSS. Choose a client architecture based on its threat model rather than declaring one storage mechanism universally safe.

Code example

This minimal API allows one illustrative frontend origin. Replace it with actual trusted origins and methods:

```
var builder = WebApplication.CreateBuilder(args);
builder.Services.AddCors(options =>
{
    options.AddPolicy("Frontend", policy =>
        policy.WithOrigins("https://app.example.com")
              .WithMethods("GET", "POST")
              .WithHeaders("Content-Type", "Authorization"));
});

var app = builder.Build();
app.UseRouting();
app.UseCors("Frontend");
app.MapGet("/api/public-example", () => Results.Ok(new { message = "Hello" }));
app.Run();
```

The sample deliberately does not enable credentials or configure authentication. In an application using both, place CORS appropriately after routing and before authorization, configure authentication/authorization separately, and test real preflights and denied origins.

Likely interview questions

1. What defines an origin, and what does CORS actually control?
2. When does a browser send a preflight request?
3. Why does CORS not secure an API against direct non-browser clients?
4. Why is wildcard origin access incompatible with credentialed CORS?
5. How do CORS and CSRF differ?
6. What changes when TLS terminates at a reverse proxy?
7. Why must forwarded headers be trusted only from known proxies?
8. What additional controls would you apply to a financial API beyond HTTPS?

Common pitfalls / misconceptions

- Treating CORS as authentication, authorization, or complete CSRF protection.
- Allowing arbitrary origins to access credentialed responses.
- Trusting forwarded headers from the public internet.
- Assuming TLS protects tokens stored in logs or prevents application-level authorization flaws.

Cheat-sheet recap

- CORS is a browser cross-origin response-access mechanism; it is not an API access-control system.
- Configure exact trusted origins and necessary methods/headers; test preflight and credentials.
- TLS protects transport; trusted proxy configuration and application security still matter.

Day 35 — Swagger + API versioning + review

Month 2 — Full-Stack Depth, Week 1 — ASP.NET Core + Web API (Week 5 overall).

Goal: Describe and document an API accurately, plan compatible evolution, and review authentication and security tradeoffs.

OpenAPI and Swagger

OpenAPI is a specification for describing HTTP APIs: paths, methods, parameters, request and response schemas, security schemes, and other metadata. Swagger is a family of tools that can generate, display, or consume OpenAPI descriptions; Swagger UI is one interactive documentation interface. An OpenAPI document is not the same as the running server, a test suite, or an authorization system.

Generate documentation from endpoint metadata and enrich it with descriptions, examples, validation constraints, status codes, and security requirements. Keep it aligned with actual behavior through contract or integration tests. A documented bearer scheme tells clients how to provide a token; the API must still validate tokens and enforce permissions. Avoid exposing internal-only endpoints, sensitive examples, or interactive production access without a deliberate policy.

ASP.NET Core versions and packages differ: modern .NET provides built-in OpenAPI document generation through `Microsoft.AspNetCore.OpenApi`, while interactive Swagger UI commonly requires additional

tooling. Verify the framework and package versions rather than assuming every template registers the same services.

API versioning and compatibility

Versioning provides a way to evolve a contract when changes cannot safely be made compatible. Common approaches include URI segments such as `/api/v1/deposits`, query parameters, headers, or media types. URI versions are visible and easy to route; header or media-type versions can keep resource URIs stable but require careful client documentation, observability, and cache variation. No single approach is universally best.

Adding an optional response field may be compatible for tolerant clients but can still break strict schema consumers; renaming or removing a field, changing semantics, or narrowing accepted inputs can be breaking. Test actual clients and contracts rather than assuming compatibility. Maintain versions for a documented support period, measure usage, communicate deprecation and sunset dates, and provide migration guidance. Security fixes may need to reach every supported version.

Versioning libraries can help with route selection, API exploration, and version-specific OpenAPI documents. They do not make behavior compatible automatically. Avoid silently defaulting a client to an unintended version when the requested version is unsupported; return a clear, documented response.

Code example

For a .NET 10 minimal web project, add the `Microsoft.AspNetCore.OpenApi` package compatible with the target framework, then use:

```
var builder = WebApplication.CreateBuilder(args);
builder.Services.AddOpenApi();

var app = builder.Build();
if (app.Environment.IsDevelopment())
    app.MapOpenApi();

app.MapGet("/api/v1/deposits/{id:int}", (int id) =>
    Results.Ok(new { id, amount = 100m }))
    .WithName("GetDepositV1");

app.MapGet("/api/v2/deposits/{id:int}", (int id) =>
    Results.Ok(new { id, amount = 100m, currency = "INR" }))
    .WithName("GetDepositV2");

app.Run();
```

The generated OpenAPI document is available in development at the framework's default document endpoint, typically `/openapi/v1.json`. The `v1` in that default **document name** is not automatically the API route version; this example places both routes in one document. It does not add Swagger UI, persistence, authentication, or a dedicated API-versioning library.

Week review

- **REST and HTTP:** Identify resources, choose methods and status codes by semantics, and design caching, conditional updates, and retries deliberately.
- **Authentication and authorization:** Validate the correct access token using trusted issuer keys and expected audience; then check endpoint permissions and resource ownership.
- **Errors and transport:** Return safe, consistent failures; use HTTPS, trusted proxy configuration, and browser-appropriate CORS rather than relying on any one control.
- **Evolution:** Keep OpenAPI accurate, test compatibility, monitor usage, and communicate migrations.

Real-world tie-in: an auth/security decision

Describe an authentication or security decision from **actual production work**. Select a decision you can substantiate: the problem and threat, options considered, chosen control, your own contribution, tradeoff, and how the team verified the outcome. Protect confidential details. If you did not make the decision personally, distinguish your role from the team's; if you cannot recall a suitable example, do not invent one.

Model answer shape: “In [actual system], we needed to protect [real resource] against [specific risk]. We considered [real alternatives] and chose [actual mechanism] because [constraints and tradeoff]. My contribution was [your role]. We verified it through [actual test or monitoring]. The remaining limitation was [honest limitation], and I would improve [specific next step].”

Likely interview questions

1. What is the difference between OpenAPI, Swagger UI, and runtime authorization?
2. How do you keep generated API documentation consistent with implementation?
3. What are the tradeoffs of URI, header, and media-type API versioning?
4. Why can an apparently additive response change still break some clients?
5. How would you deprecate and retire an API version safely?
6. How do authentication, endpoint policy, and resource ownership combine in a secure request?
7. Why are CORS and HTTPS insufficient as the only API security controls?
8. Describe a real security decision, including alternatives, your contribution, and verification.

Common pitfalls / misconceptions

- Treating an OpenAPI security declaration as enforcement.
- Confusing an OpenAPI document's name with an endpoint's API version.
- Retiring a version without usage data, a support policy, and migration communication.
- Inventing production security decisions or overstating personal ownership.

Cheat-sheet recap

- OpenAPI describes the contract; Swagger tools can present or consume it.
- Version based on compatibility needs; document and test each supported contract.
- Secure the runtime independently of the documentation.
- Real incident or decision: context → threat → alternatives → decision → verification → tradeoff.